LP-310: Universal Liquidity

One Router, Every Source: Best-Execution Settlement Across Native AMM Pools, Signed-Quote Market Making, and External Venues

Zach Kelling

Lux Industries

June 2026

Abstract

Universal Liquidity is the order-execution layer of the Lux exchange. It unifies three structurally different liquidity sources behind one router and one atomic settlement contract: the Network's native automated market makers (AMM V2/V3 contracts and the V4 native precompile at `0x9999`), a signed-quote private market maker (PMM) that prices against and hedges on external venues, and external on-chain DEX liquidity used only as a last resort. A single request yields a single best-execution fill that succeeds in full or reverts in full.

The paper formalizes four things. First, the *signed-quote request-for-quote (RFQ)* protocol: the market maker — not the user — signs the price, so "you will receive y" is a cryptographic commitment enforced on-chain, and a short, on-chain-checked expiry eliminates the free-option ("last-look") leak that defeats naive RFQ designs. Second, the *universal router*: best execution is *decided off-chain and settled on-chain*, never resolved inside the settlement transaction. Third, the *off-chain/on-chain dividing line* as a design law — a backend exists only where off-chain state (price, inventory, hedge) is unavoidable, which is why swap requires a backend and liquidity provision requires none. Fourth, the *post-quantum* unforgeability of the quote commitment, inherited from the Lux PQ signature stack (LP-073 Pulsar, ML-DSA-65; [**?**, **?**]).

Section **??** maps the design to the deployed system: `luxfi/dex`, the `@luxamm` SDKs, the V2/V3 factory and position-manager contracts live on Lux mainnet, the V4 precompile, and the `exchange-api` settlement relayer.

Contents

1 Introduction

A trader who wants to convert one asset into another should not have to know *where* the liquidity comes from. Whether the best price at that instant sits in a Lux native AMM pool, in the inventory of a market maker hedging on a centralized venue, or in some external DEX, the trader's experience should be identical: ask for a price, receive a firm quote, sign once, and obtain exactly that price or nothing. We call this property *universal liquidity*, and this paper specifies the protocol that delivers it.

1.1 Three sources, one fill

Lux exposes three structurally different sources of liquidity:

(i) **Native on-chain liquidity** — the V2 (constant-product) and V3 (concentrated-liquidity) constant-function AMM contracts, whose price is a pure function of on-chain reserves, together with the D-Chain limit-order-book venue whose fills settle on-chain through the `0x9999` DEX receipt-settlement precompile (a V4-shaped ABI, *not* itself a constant-function AMM). Execution is trustless and atomic.

(ii) **Signed-quote market maker (PMM)** — a private market maker that prices a pair against an external reference (e.g. a centralized exchange), adds a spread, and commits to a firm quote with a short lifetime. After filling on-chain it hedges off-chain to restock. Pricing depends on *off-chain* state (reference price, inventory) and so cannot be a pure on-chain function.

(iii) **External DEX** — third-party on-chain liquidity, used only when neither of the above can fill, and only to avoid a hard revert.

A universal-liquidity request returns a *single* fill drawn from whichever source(s) give the trader the best execution, settled in one atomic transaction. The waterfall the operator runs is, in one line:

native pools first → if insufficient, the PMM → else external DEX → else revert.

Section **??** examines when "ours first" is the right ordering and when it silently costs the trader, and resolves the tension into a single, stated rule.

1.2 What makes it hard

The naive picture — "ask a backend for a price, then trade" — hides three failure modes that this paper is built to close.

The price must be a commitment, not a hint. If the backend merely *tells* the user a price and the settlement contract does not enforce it, the realized fill can differ from the quote. The fix is that the market maker *signs* the quote; the settlement contract verifies that signature and guarantees the signed output amount or reverts (§3).

A firm quote with no expiry is a free option. A signed quote that never expires lets the holder wait: execute when the market has moved in their favor, discard it otherwise. This "last-look in reverse" bleeds the market maker. The fix is a short, on-chain-checked expiry baked into the signed order (§3, Thm. 3.2).

Best execution must not be computed on-chain. Resolving the waterfall inside the settlement transaction — calling external pools mid-fill to discover the best route — is gas-heavy, reentrancy-prone, and non-deterministic. The fix is a strict separation: *decide off-chain, settle on-chain* (§**??**).

1.3 The dividing line

The single most clarifying idea in this design is a law about where state lives:

Principle 1.1 (Backend only for unavoidable off-chain state)**.** *A server-side service is justified for an operation if and only if that operation depends on state that is not on-chain. Swap-via-market-maker depends on the reference price, on inventory, and on the hedge — all off-chain — so it requires a backend. Liquidity provision (mint/burn of an AMM position) is a pure on-chain interaction — no quote, no inventory, no hedge — so it requires* no *backend at all.*

Principle 1.1 decomplects the system: the `exchange-api` relayer is the swap path's quote-and-settle engine, and liquidity provision is built entirely client-side from the `@luxamm` SDK and the deployed contract addresses, depending on nothing external (§**??**). Re-introducing a "liquidity service" that merely assembles add/remove calldata would add a trusted middleman that re-derives what the SDK already produces; the design forbids it.

1.4 Post-quantum from the signature up

The quote commitment is only as strong as the signature that binds it. Universal Liquidity inherits the Lux post-quantum signature stack: the market-maker key is an ML-DSA-65 key, the same scheme that anchors validator identity and the Pulsar threshold layer (LP-073, LP-020; [**?**, **?**]). A quantum adversary that breaks classical ECDSA cannot forge a quote, so "you will receive y" remains a binding commitment in the post-quantum setting (§**??**).

2 Liquidity Sources and the State Boundary

We formalize the three sources by the only property that matters to the router: what state their price is a function of, and therefore where that price can be computed.

2.1 Native AMM: price is a pure function of reserves

A constant-function market maker prices a trade as a deterministic function of its on-chain reserves R. For V2 the invariant is the constant product $x \cdot y = k$; a swap of Δ_{in} (net of fee ϕ) returns

$$\Delta_{\text{out}} = \frac{y \cdot (1-\phi)\,\Delta_{\text{in}}}{x + (1-\phi)\,\Delta_{\text{in}}}.$$

For V3 the same invariant holds piecewise over each initialized tick range, with liquidity L concentrated between price bounds; the realized output is the integral of L across the ticks the trade crosses ([**?**]). The native constant-function AMM comprises these V2 and V3 contracts only; the `0x9999` precompile is *not* a constant-function AMM but the DEX receipt-settlement conduit for the D-Chain limit-order-book venue, whose price is set by the order book rather than by on-chain reserves.

The decisive property is purity: given the block's reserves, the output is exact and reproducible by any party. No off-chain information is required to quote a native pool, so a native-pool quote is trustless and needs no signature.

2.2 Signed-quote market maker: price depends on off-chain state

The PMM prices a pair $A \to B$ as

$$y \;=\; \mathrm{ref}_{A\to B} \cdot (1 - s) \cdot x,$$

where $\mathrm{ref}_{A\to B}$ is an external reference price and s is the spread. Both inputs are off-chain: the reference comes from a venue's order book, and the spread is set against inventory and volatility. The quote therefore *cannot* be a pure on-chain function — it is a signed assertion by the market maker, valid until a stated expiry, and made enforceable on-chain only by verifying the signature (§3).

After an on-chain fill the PMM is left with an inventory imbalance ($-x$ of A, $+y$ of B relative to target) and restocks by trading the opposite direction on the external venue. The spread s is the compensation for the risk taken between quote and completed hedge (§**??**).

2.3 External DEX: trustless but last

External on-chain pools are, like native AMMs, pure functions of their reserves and require no signature. They differ only in policy: they are consulted last, purely to convert a would-be revert into a fill, and at whatever price they offer. They introduce no new trust assumption — the settlement transaction either obtains the required output from them or reverts.

2.4 Summary: the boundary as a table

Table 1 states the property that drives every later decision. The router (§**??**) reads off this table; the backend/no-backend split (Principle 1.1) is exactly the "off-chain state?" column.

Table 1: The three sources by state dependence.

Source	Price is a function of	Off-chain state?	Needs signature?
Native AMM V2/V3/V4	on-chain reserves	no	no
Signed-quote PMM	reference + inventory + spread	yes	**yes**
External DEX	on-chain reserves	no	no

3 Signed-Quote RFQ and Atomic Settlement

The request-for-quote path is where universal liquidity earns its correctness. We give the order object, the five-step lifecycle, the on-chain checks, and the two theorems that make the path safe for both parties.

3.1 The signed order

Definition 3.1 (Quote order)**.** *A quote order is the tuple*

$$\mathcal{O} = (\mathit{tokenIn},\ \mathit{tokenOut},\ \mathit{amountIn},\ \mathit{amountOut},\ \mathit{expiry},\ \mathit{nonce},\ \mathit{taker}),$$

signed by the market maker under its ML-DSA-65 key to produce $\sigma_{\mathrm{MM}} = \mathrm{Sign}_{sk_{\mathrm{MM}}}(\mathcal{O})$*. The market maker's commitment is precisely: "a transaction submitted before* expiry *that delivers* amountIn *of* tokenIn *will receive at least* amountOut *of* tokenOut*."*

It is the *maker* who signs the price. The taker authorizes the movement of their own funds separately — via a Permit2 signature ([**?**]) — which is a gasless approval, not a price commitment. This separation is the crux: the maker is bound to the price, the taker is bound to the spend, and neither can unilaterally alter the other's obligation.

3.2 Lifecycle

1. **Quote.** The taker's frontend requests a price for *amountIn* of $A \to B$. The backend reads the reference price, adds spread, *reserves* the required B inventory (§3.4), and returns a signed order $\mathcal{O}$ with a short expiry.
2. **Sign (taker).** The taker signs a Permit2 message authorizing the settlement contract to pull *amountIn* of A. This is off-chain and gasless.
3. **Submit.** The backend relays one transaction to the settlement contract carrying $(\mathcal{O}, \sigma_{\text{MM}}, \mathit{permit2}_{\text{taker}})$. The relayer pays gas; its cost is folded into the spread.
4. **Settle (atomic).** The contract performs the checks below and, if all pass, pulls A from the taker and delivers *amountOut* of B from maker inventory in the same call — fully or not at all.
5. **Hedge.** The backend restocks by trading $A \to B$ on the external venue to flatten the inventory move from step 4.

3.3 On-chain checks (the settlement predicate)

The settlement contract accepts $(\mathcal{O}, \sigma_{\text{MM}}, \mathit{permit2})$ iff all of:

(C1) $\text{Verify}_{pk_{\text{MM}}}(\mathcal{O}, \sigma_{\text{MM}})$ holds — the maker signed this exact order;

(C2) $\mathit{block.timestamp} \le \mathcal{O}.\mathit{expiry}$ — the quote is live;

(C3) $\mathcal{O}.\mathit{nonce}$ is unused, then marked used — no replay;

(C4) the Permit2 pull of *amountIn* from the taker succeeds; and

(C5) maker inventory holds $\ge$ *amountOut* of *tokenOut*.

Any failure reverts the entire transaction, restoring both balances.

3.4 Inventory reservation

A quote must be backed by inventory at the moment it is issued, not hoped for at settlement. The backend reserves *amountOut* of B against the live quote and releases the reservation on expiry or fill. Without reservation a maker can sign more deliverable output than it holds; check (C5) then turns oversubscription into user-visible reverts. Reservation moves the failure to quote time, where it is invisible to the taker (the maker simply quotes a smaller size or declines).

3.5 The two safety theorems

Theorem 3.1 (Quote integrity)**.** *If the settlement transaction succeeds, the taker receives at least $\mathcal{O}.\mathit{amountOut}$ of tokenOut for exactly $\mathcal{O}.\mathit{amountIn}$ of tokenIn.*

Proof. Success requires (C1)–(C5). (C1) binds the realized amounts to the signed $\mathcal{O}$; (C4) fixes the input at *amountIn*; the delivery in step 4 transfers *amountOut* gated by (C5). Atomicity makes the transfer all-or-nothing, so a successful transaction realizes exactly the signed amounts. The maker cannot substitute a worse price because any other amounts produce a different $\mathcal{O}'$ for which σ_{MM} fails (C1). □

Theorem 3.2 (No free option)**.** *A taker holding a signed order $\mathcal{O}$ cannot execute it after $\mathcal{O}.\mathit{expiry}$.*

Proof. Execution requires (C2), $block.timestamp \leq \mathcal{O}.expiry$, checked against the inclusion block's timestamp. After expiry every inclusion fails (C2) and reverts. The window in which the quote is exercisable is therefore bounded by $\mathcal{O}.expiry - t_{\text{quote}}$, which the maker sets to the order of seconds. The taker cannot hold the quote as a free option on adverse price movement. □

3.6 Hedge risk is the spread's job

Between the on-chain fill (time T) and the completed external hedge (time $T + \delta$) the reference price drifts. The maker's profit on a filled order is

$$\Pi \;=\; \underbrace{s \cdot \text{ref}_T \cdot x}_{\text{spread captured}} \;-\; \underbrace{\left(\text{ref}_{T+\delta} - \text{ref}_T\right) \cdot x}_{\text{hedge slippage}} \;-\; g,$$

with g the relayer gas. The spread s must dominate the expected hedge slippage over the latency δ plus gas; for volatile pairs this means a wider s or pre-held inventory rather than quote-then-hedge. Two operational invariants follow:

- **Hedge failure is not fill failure.** The on-chain fill is final the instant it confirms; if the subsequent external hedge fails (venue outage, insufficient balance) the maker is left directionally exposed and must retry or hedge on a secondary venue. The settlement path must never make the user's fill contingent on the maker's hedge.
- **Spread is priced for δ, not for 0.** A spread computed as if the hedge were instantaneous systematically loses on the tail of δ; the maker prices the latency, not the mid.

4 The Universal Router

The router is the component that turns three structurally different liquidity sources (§2) into one best-execution fill. Its governing principle is a single design law, stated here and justified through the rest of the section.

> **Router Law.** *Best execution is decided off-chain and settled on-chain. Price discovery, quote aggregation, and source selection happen before any transaction is sent; the settlement transaction only* verifies and executes *a decision already made.*

The reason is economic, not merely engineering. Resolving best execution *inside* the settlement transaction — iterating pools, pulling signed quotes, comparing venues on-chain — would (i) pay gas to discover a price the router already knows, (ii) expose the in-flight comparison to the mempool, reintroducing exactly the front-running the signed-quote RFQ (§3) exists to remove, and (iii) make the fill's gas cost a function of how many sources were consulted. Deciding off-chain and settling on-chain makes settlement cost constant and the executed price a cryptographic commitment rather than the outcome of an on-chain race.

4.1 The waterfall

Given a request to trade size q of pair (A, B), the router evaluates the sources in a fixed precedence and returns the single fill, or split of fills, that maximizes the trader's received amount net of fees and gas:

W1. Native first. Quote the native venues — the constant-function AMM (V2/V3) and the D-Chain limit-order book, whose fills settle atomically through the `0x9999` precompile. These are pure functions of on-chain state (§2), so their quotes are exact and require no counterparty: a native fill cannot be reneged.

W2. Signed quote when it improves. Request a firm signed quote from the private market maker (PMM). If, after its spread, the PMM price beats the native price for size q, route to the PMM under the RFQ settlement predicate (§3). The PMM prices against external reference venues and hedges after filling, which is how *global* off-chain liquidity reaches an on-chain trader.

W3. Split when the book is thin. For sizes that walk the native book past the PMM's marginal price, the router splits: fill the first tranche natively and the remainder against the signed quote, still in one atomic settlement (§**??**).

W4. External DEX only to avoid a revert. If neither native nor PMM can fill — an exotic pair, a venue outage — the router falls back to third-party on-chain liquidity. This is last precisely because it is trustless-but-uncompetitive: it exists so a request degrades to a worse price rather than to a failure.

[Monotone best execution] For any request, the router's returned amount is $\geq$ the amount from any single source in isolation. Adding a source can only weakly improve the fill (a source that never improves is never selected), and the split in **W3** dominates any single-source fill whenever the sources' marginal prices cross within q.

4.2 Global aggregation

"Universal" is not confined to one chain. Two mechanisms extend the waterfall across the whole Lux family and beyond:

- **Cross-L1 liquidity.** Each sovereign Lux L1 runs its own native venues; the router quotes pools and books on *sibling* L1s and settles cross-chain through the bridge, whose transfers are attested by the same dealerless threshold-MPC group that custodies assets. A request on one tenant's venue can therefore be filled against liquidity resident on another, without either operator taking custody of the other's assets.

- **Off-chain global reference.** The PMM leg (W2) is the conduit for centralized-venue depth: the maker prices against global reference markets, commits a firm on-chain quote, and hedges off-chain afterward. The trader receives global-market pricing with on-chain, non-custodial, atomic settlement — the property no purely-on-chain AMM and no purely-off-chain OTC desk delivers alone.

4.3 One atomic settlement

Whatever the router decides — one source or a split across several — the result settles in a *single* transaction that succeeds in full or reverts in full. The settlement contract validates each leg's predicate (native pool invariant for AMM legs; the signed-quote predicate of §3 for the PMM leg; the pool constraint for an external leg), moves all legs, and emits one settlement receipt through the `0x9999` conduit.

[All-or-nothing] A router fill has no partial-execution state visible to the trader: either every leg settles in the same block and the trader receives the aggregate quoted amount, or the transaction reverts and no leg settles. There is no intermediate state in which one leg filled and another did not.

Atomicity is what makes the split safe: without it, a two-source fill could leave the trader having paid for one leg while the other reneged. With it, the router can compose native depth and signed-quote depth as freely as if they were one book — which, from the trader's single-signature vantage, they are.

4.4 What the settlement transaction is not

The router deliberately keeps three things *out* of the settlement transaction: price discovery (done in the quote phase), counterparty solicitation (the PMM signed its quote before the transaction was built), and source iteration (the split was computed off-chain). What remains on-chain is verification and movement — a bounded, constant-shaped computation. This is the concrete meaning of the Router Law, and the reason a universal fill costs the same gas whether it drew from one source or four.

5 Liquidity Provision

A router is only as good as the depth behind it. This section specifies how each source is *supplied*, and — the operative question for a white-label operator standing up a new venue — how a venue reaches competitive depth on day one rather than after months of organic bootstrap.

5.1 Native AMM: passive, permissionless depth

The constant-function AMM is supplied by liquidity providers who deposit a reserve pair. V2 spreads capital uniformly along the price curve; V3 concentrates it in a chosen range, so the same capital quotes tighter around the active price at the cost of going idle outside the range ([**?**]). Provision is permissionless and non-custodial: an LP's capital sits in the pool contract, priced by the invariant, and is withdrawable at will. This is the always-available floor of the waterfall (**W1**) — it never reneges and never goes offline, but its price is mechanical and its depth is whatever LPs have committed.

5.2 D-Chain order book: active, professional depth

The D-Chain limit-order book is supplied by makers posting resting orders into the matching engine. Unlike the AMM, book depth reflects makers' *views*, not a fixed curve, and can be far tighter around the mid. The engine matches deterministically at hardware speed — a mean of 169 ns per match, with millions of orders per second of throughput and bit-exact CPU/GPU parity — so a professional maker can quote, cancel, and re-quote at a cadence competitive with a centralized venue, while fills still settle on-chain through `0x9999`. Determinism is what makes the book auditable: the match sequence is reproducible, a requirement a regulated venue cannot waive.

5.3 Signed-quote PMM: global depth without on-chain inventory

The private market maker supplies the deepest and most price-competitive leg without pre-committing capital to a pool. It prices a pair against external reference venues, commits a firm on-chain quote for a short window, fills against the trader, and hedges off-chain to restock (§3). Because its inventory lives off-chain, the PMM can quote size far beyond any on-chain reserve, and because it prices against global markets, its quotes track the global mid. The cost is the structural one made precise in §3: the maker, not the user, signs the price; a short on-chain expiry removes the last-look free option; and the spread is priced for hedge latency, not for the instantaneous mid.

5.4 Bootstrapping a new venue

The three mechanisms compose into a concrete answer to the cold-start problem that kills most new venues: an empty book quotes nothing, and no trader arrives before there is depth, and no maker posts depth before there are traders. A venue built on Universal Liquidity breaks the loop from the first block:

B1. Plug in a PMM on day one. The operator (or a partnered market maker) points a signed-quote PMM at the pairs it wants live. The venue immediately quotes global-market pricing at professional size, *before* any passive LP or resting order exists — the PMM's off-chain inventory is the initial depth.

B2. Seed native pools in parallel. Passive V3 liquidity is added around the active price to provide the trustless floor and to capture flow the PMM does not want.

B3. Borrow sibling depth. The cross-L1 aggregation of §**??** lets the new venue quote against liquidity already resident on other Lux L1s, so it is not starting from zero even for its native legs.

B4. Let the book fill in. As flow arrives, professional makers post resting D-Chain orders, and organic V3 provision deepens. The venue transitions from PMM-anchored to book-anchored depth without any discontinuity in the trader's experience.

Remark 5.1. *For an operator reselling a regulated venue, this is the decisive property. The hardest part of launching an exchange is not the software — it is having a price and size to show on the first trade. Universal Liquidity makes day-one depth a* configuration *(point a PMM at the pairs, seed some V3, enable sibling aggregation) rather than a multi-quarter liquidity-mining campaign. The same mechanism serves crypto pairs and tokenized-securities pairs identically, so a single venue lists both with competitive depth from launch.*

6 Security

Universal Liquidity composes several trust boundaries — an on-chain settlement contract, an off-chain market maker, a router that decides before it settles, and cross-chain aggregation. This section states the threat model and shows how each attack the composition invites is closed by a mechanism already introduced.

6.1 Threat model

We assume an adversary who can (C1) observe every pending transaction and every public quote; (C2) submit transactions at arbitrary size and fee, and influence ordering up to the consensus Byzantine bound; (C3) operate as, or collude with, a market maker; and (C4) run flash-loan capital. We do *not* assume the adversary can forge signatures (ECDSA/BLS/ML-DSA-65), break the AMM invariant, or violate the atomicity of a single transaction.

6.2 Front-running and the free option

A naive RFQ leaks a free option: if the *user* signs and the maker chooses whether to fill ("last look"), the maker fills only when the market has moved in its favor and walks otherwise, and an observer can front-run the pending fill. Universal Liquidity closes both:

- **The maker signs, not the user** (§3). "You will receive y" is the maker's cryptographic commitment, enforced by the on-chain settlement predicate. The maker cannot renege on a fill the user chooses to take within the quote's life.
- **Short, on-chain-checked expiry.** The quote is valid only until a block-timestamp bound the contract verifies. The free-option window is bounded to the quote lifetime, and the spread (§3) prices that residual exposure explicitly (Theorem 3.2).
- **Off-chain decision.** Because best execution is resolved before the transaction is built (Router Law, §**??**), there is no in-flight price comparison in the mempool for an adversary to sandwich.

6.3 Atomicity and partial-fill attacks

The all-or-nothing property (§**??**) is a security property, not just a UX one. Without it, a split fill could be attacked by causing one leg to fail after another settled, leaving the trader mispriced. With single-transaction atomicity, every leg's predicate is checked and every leg moves in the same block, or none does. A router that composes native and signed-quote legs is therefore no more attackable than a single-source fill.

6.4 Inventory double-spend

A PMM that signs two quotes against the same inventory and has both settle would be over-committed. The inventory-reservation mechanism (§3.4) makes a signed quote consume a reservation that the settlement predicate checks, so a given unit of maker inventory backs at most one settleable quote at a time. Over-commitment is prevented on-chain, not by the maker's honesty.

6.5 Reference-price and oracle manipulation

The PMM prices against external reference venues; an adversary who can move that reference (thin external market, flash-loan-funded push) can skew the quote. Two bounds contain this: the maker owns its spread and widens it when reference confidence drops, and the router never routes to the PMM when the native (manipulation-resistant, on-chain-reserve) price is better — so a manipulated PMM quote loses to the native floor rather than executing. The native AMM/book legs are priced by on-chain state and are not oracle-dependent.

6.6 Non-custodial settlement and pre-trade privacy

Two platform properties reinforce the protocol. First, settlement is *non-custodial*: legs move under the dealerless threshold-MPC custody of the underlying L1, so no router, maker, or venue operator ever holds material sufficient to move a trader's assets unilaterally — the exchange coordinates a threshold, it does not take custody. Second, where an operator enables it, order intent can be submitted encrypted and decrypted only at block construction (the encrypted-mempool path), removing the pre-trade information leak that funds front-running at the source rather than merely bounding it.

[Composition safety] Under the threat model above, a universal fill is safe against front-running (maker-signed, off-chain-decided), free-option extraction (short on-chain expiry, spread-priced), partial-fill manipulation (single-transaction atomicity), inventory double-spend (on-chain reservation), and reference manipulation (native floor dominates a skewed quote). Each guarantee reduces to a mechanism enforced on-chain, not to the honesty of an off-chain party.

7 Current Status

This section states, without embellishment, what is shipped versus specified. An operator evaluating the venue should be able to separate the running substrate from the protocol layered on it.

Component	Status	Evidence / location
`0x9999` settlement precompile	**Shipped, live**	Mainnet C-Chain; a maker/taker fill settled on-chain with an at
D-Chain CLOB matching engine	**Shipped**	`luxfi/dex`: 169 ns avg match (p50 125 / p99 292 ns), 5.91M ord
AMM V2 / V3 contracts	**Shipped**	`luxfi/standard` constant-function pools
Signed order / inventory reservation	**In build**	Signed-order path and GPU-verified orders exist; the full RFQ
Universal router (waterfall + split)	**In build**	Off-chain quote aggregation and the atomic multi-leg settlemen
Cross-L1 aggregation	**Spec**	Depends on the shipped bridge; the router's sibling-quote path
Encrypted-intent submission	**Spec**	The FHE precompile is shipped; the mempool glue is a separat

7.1 The settled fill, in full

The load-bearing claim — that atomic, real-time, non-custodial on-chain settlement is a running primitive rather than a design — is evidenced by a live mainnet fill. A maker posted a bid; a taker sold 10 units of the base asset into it at price 1.1; the settlement receipt carried the on-chain fill event with output amount 11, together with both ERC-20 transfer legs, in a single accepted transaction, conserving value exactly ($10 \times 1.1 = 11$). The fill crossed the fresh bid rather than a stale resting order, confirming price priority. Everything in §3–§**??** composes *above* this settled primitive.

7.2 Honest gaps

The router and the full RFQ predicate are the active build surface; until they land, the venue runs on the shipped native legs (AMM + book + `0x9999`) with the signed-quote PMM leg maturing. Cross-L1 aggregation and encrypted intent are specified and depend on shipped substrate (bridge, FHE precompile) but are not yet wired into the router. None of these gaps affect the guarantees of the shipped native path; they extend its reach and depth.

8 Conclusion

Universal Liquidity is the order-execution layer that lets a Lux venue show one price, drawn from everywhere, and settle it in one atomic, non-custodial transaction. The design reduces to a small number of load-bearing choices:

- **One fill from three sources.** Native AMM and order-book depth, a signed-quote market maker pricing against global venues, and external DEX liquidity as a last resort, unified behind a router that returns a single best-execution result (§**??**).
- **Decide off-chain, settle on-chain.** Price discovery and source selection happen before the transaction; settlement only verifies and moves. This makes execution a cryptographic commitment rather than an on-chain race, and keeps gas constant regardless of how many sources were consulted (Router Law).
- **The maker signs the price.** The signed-quote RFQ turns "you will receive y" into an on-chain-enforced commitment and, with a short on-chain expiry, removes the last-look free option that defeats naive RFQ (§3, §**??**).
- **Global reach, non-custodial settlement.** Cross-L1 aggregation and the PMM's off-chain hedging bring global depth to an on-chain trader, while every leg settles under dealerless threshold custody — no venue, router, or maker ever takes possession (§**??**, §**??**).

For an operator reselling a regulated venue, the payoff is the property that usually takes a new exchange quarters to earn: **competitive depth on day one**. Point a market maker at the

pairs, seed passive V3, enable sibling aggregation, and the venue quotes global-market pricing at professional size from its first block — for crypto pairs and tokenized-securities pairs alike, on the same book, under the same custody root (§**??**). Combined with the ATS/BD/TA services and the sovereign-L1 substrate they run on, Universal Liquidity is what turns "a chain" into "a venue an operator can launch and sell."

The settlement primitive at the bottom of the stack is live on mainnet today (§**??**); the router and the full RFQ predicate are the active build surface above it. The direction is fixed and the substrate is running: liquidity that is deep, global, best-execution, atomic, and non-custodial — exposed to the trader as a single signature.

References

[1] Lux Industries. *LP-020: Quasar Consensus*. Lux Papers, 2026.

[2] Lux Industries. *LP-073: Pulsar — Module-LWE Threshold Signatures*. Lux Papers, 2026.

[3] Lux Industries. *LP-308: QuantumSwap — A Post-Quantum AMM Protocol*. Lux Papers, 2026.

[4] Uniswap Labs. *Permit2: a token-approval contract for signature-based, batched approvals*. 2022.

[5] H. Adams et al. *Uniswap v3 Core*. 2021.

[6] Lux Industries. *LP-201: ZAP Universal Wire Protocol*. Lux Papers, 2026.